Where and What: Executable Oracles as Acceptance Gates and Repair Feedback for LLM-Generated Mechanism Simulators

Phi Hai Hoang[1*]

[1*]FPT School of Business and Technology (FSB), FPT University, Ho Chi Minh City, Vietnam.

Corresponding author(s). E-mail(s): haihpaws@gmail.com;

Abstract

Large language models can write interactive 3D simulators of machines from a short description. Such programs run and look plausible, yet they can be kinematically wrong, and current pipelines accept them after a vision-language model has looked at screenshots. We present an executable test oracle for generated mechanism simulators. It samples the page's pure rendering function over time and checks structural invariants, the dimensions the simulator declares, engineering design rules, and the kinematic solution computed from those declared dimensions without hand-written per-task specifications. Using eight mechanisms, 88 generated simulators and two models of different size, we ask which errors occur, what the oracle and a vision-language reviewer detect, whether the oracle can gate iterative repair, and how specific its feedback must be. Under free-form specifications, 20 of 80 simulators were wrong, mostly in design knowledge such as misassigned published link lengths; visual review missed invisible faults such as an impossible gear assembly. In repair with at most three rounds, feedback naming the faulty element together with its correct value fixed 28 of 28 faulty cases; naming the element only, 15 of 28; naming the failed check and its magnitude, 11 of 28; self-refinement without feedback, 7 of 28; and the visual reviewer in the loop, 5 of 22. The information needed depends on the repairing model: on design-knowledge errors the smaller model succeeded only when given the correct value (13 of 13 versus 0 of 13), while the larger one needed only the location (7 of 7).

Keywords: test oracle, LLM code generation, automated program repair, simulation verification and validation, metamorphic testing, vision-language models

1 Introduction

Large language models (LLMs) are increasingly used to produce not only snippets of code but complete, runnable artefacts: interactive scientific demonstrations [1], physics animations [2, 3], articulated 3D objects [4, 5], CAD models [6] and whole 3D worlds [7]. One instance of this trend is the *mechanism simulator*: a single web page in which a machine (a linkage, a gear train, a cam, a walking leg) is drawn in 3D and moves, with live numbers that explain it. Tools that generate such pages from a one-paragraph request are already available to end users [8]. They are attractive for engineering education, design communication and, increasingly, as lightweight digital twins.

Such a page has a peculiar correctness problem. It almost always loads and animates, and it usually looks right. Whether it is *kinematically* right—whether the parts stay connected, follow the motion the machine actually has, and respect

the design rules that make the machine work—cannot be seen in a handful of frames and is not covered by any unit test, because there are none. In the generation pipelines we are aware of, the acceptance step is a vision-language model (VLM) that inspects screenshots [3, 5, 6]. This is a mechanised version of the oldest simulation-validation technique, watching the animation [9], with the same weakness: plausible-looking motion is accepted.

Consider Theo Jansen's walking leg [10]. Its eleven link lengths, the "holy numbers", make the foot trace a path with a long flat bottom. In our study, both models we tested remembered the eleven numbers but often assigned them to the wrong links. The resulting legs assemble, move smoothly and look like Jansen legs, but the flat stride shrinks from about a third of the cycle to a few percent. A VLM shown the frames reported the leg as correct in some runs and as faulty in others. When asked to re-check its own work, the generating model asserted that the lengths "match the published holy numbers" and, when told otherwise, that the checker must be wrong. An executable check that compares the declared lengths with the published ones settles the question in a fraction of a second and can also say which link is wrong and what its length should be.

This paper studies such executable oracles for LLM-generated mechanism simulators, both as an *acceptance gate* and as a source of *repair feedback*. The oracle (Section 3) needs no hand-written specification per task. A minimal output contract asks the generator to expose a pure function `render(t)`, the dimensions it chose, and named marker points on each part. From these, the oracle derives metamorphic and structural checks (determinism under reordered calls, periodicity, rigid parts, coincident joints), checks the drawing against the declared dimensions, applies engineering design rules, and compares every marker with the kinematic solution computed from the declared dimensions. When a check fails, it reports where (which link, joint or marker, at which instant) and, where a ground truth exists, what the value should be.

We address four research questions:

- **RQ1** Which errors do LLMs make when generating mechanism simulators from free-form descriptions, and how often?
- **RQ2** Which errors do the oracle and a VLM reviewer detect?
- **RQ3** Does the oracle work as an acceptance gate for iterative repair, compared with VLM review in the loop and self-refinement?
- **RQ4** How specific must the oracle's feedback be, and does the answer depend on the repairing model?

We study eight mechanisms, 88 generated simulators, 39 seeded mutants and nearly 300 repair attempts by two models of different size from the same family. The main findings are:

- Free-form specifications lead to errors in 20 of 80 simulators; detailed specifications lead to none in 8. Errors concentrate in design knowledge that must be recalled (Jansen's link lengths, Chebyshev's lambda proportions) and in assembly conditions. Mechanisms whose design rule can be stated as a relation (a cam's motion law, the Peaucellier–Lipkin conditions) produced no errors.
- The oracle and the VLM catch different error classes. The oracle catches design-knowledge and invisible errors; the VLM catches geometry outside the oracle's contract. The VLM's detection of length errors depends largely on reading the numeric panel.
- As a repair gate, oracle feedback that names the faulty element and its correct value fixes 28 of 28 faulty file-runs within two rounds. VLM review in the loop fixes 5 of 22 and self-refinement 7 of 28. VLM-driven loops stop early on wrong "correct" verdicts.
- Specificity matters in a model-dependent way. On design-knowledge errors, the smaller model succeeded only with the correct value (13/13 versus 0/13 with location only). The larger model needed only the location (7/7) (Fisher's exact test, $p = 1.3 \times 10^{-5}$).

The contributions are: (i) an executable oracle for generated mechanism simulators that needs no per-task specification and includes a guard against marker gaming; (ii) a controlled comparison of oracle, VLM and self-refinement feedback in iterative repair with oracle-gated stopping; (iii) a feedback-specificity ablation showing an interaction between the information content of feedback and the repairing model; and (iv) the full dataset

of specifications, generated and repaired simulators, mutants and oracle reports, available at https://github.com/haihpse150218/llm-mechsim-oracle and archived at https://doi.org/10.5281/zenodo.24201916.

Beyond education, the questions matter wherever simulation software is produced quickly and trusted on sight. This includes digital twins, operator training and design reviews, which are the kinds of systems in which NASA and other mission-critical organisations rely on simulation V&V [9, 11]. An acceptance gate that is cheap, deterministic and explains its verdict is a natural complement to human or model-based visual review.

Section 2 gives background, Section 3 describes the oracle, Section 4 the study design, Section 5 the results, Section 6 discusses them, Section 7 covers threats to validity, Section 8 related work, and Section 9 concludes.

2 Background

2.1 Test oracles and derived oracles

A test oracle decides whether an observed behaviour is correct [12]. Barr et al. distinguish specified oracles (from a formal specification), derived oracles (from other artefacts such as documentation, earlier versions or properties of the program), implicit oracles (crashes, exceptions) and human oracles. Metamorphic testing [13, 14] is the best-known family of derived oracles: instead of the correct output for one input, it checks a relation between outputs of related executions. Metamorphic and differential oracles are well established for scientific software [15], autonomous-driving software [16] and graphics drivers [17], where exact expected outputs are unavailable. Recent work generates oracles or assertions with neural models [18, 19] and discusses the risks of letting an LLM act as the oracle for code that an LLM wrote [20]. For LLM-generated code in particular, weak test suites accept many incorrect programs [21].

2.2 Kinematics of planar mechanisms

All mechanisms in this study are planar and driven by a single input, a crank turning or swinging at constant angular speed. The position of every joint follows from the input angle by solving loop-closure equations. For the linkages used here, each unknown joint is the intersection of two circles centred on joints already known. Each intersection has two solutions, the *assembly branches*, and a correct simulator must stay on one branch throughout the cycle. Whether a solution exists for every input angle (the linkage *assembles*) depends on the link lengths. The Grashof condition, for example, decides whether a four-bar's crank can turn fully. Beyond assembly, many mechanisms work only because of specific *design knowledge*:

- *published dimensions*, such as Jansen's eleven link lengths [10] or the proportions of Chebyshev's lambda linkage, crank a, ground $2a$, rocker $2.5a$, coupler $2.5a$ extended by a further $2.5a$ to the tracing point [22];
- *relations*, such as the Peaucellier–Lipkin conditions (equal rhombus sides, equal long arms, crank length equal to the distance between the fixed pivots), under which the output point moves on an exact straight line;
- *assembly conditions*, such as the requirement that the sum of sun and ring teeth of a planetary set be divisible by the number of equally spaced planets.

Errors in design knowledge are particularly treacherous. The mechanism still assembles and moves smoothly, but it does not do what it is for.

2.3 Generated simulators and their acceptance

We consider simulators written as one self-contained HTML page using `three.js`, as produced by the *teardown* skill for Claude Code [8]. The skill's own review loop has the model look at sheets of rendered frames and correct what it sees. The same pattern appears in research systems that use a VLM as critic of generated 3D or physics code [3–6]. In the terms of simulation V&V [9, 11], this is the *animation* technique: watching the model run to judge its behaviour. It is useful for gross errors, but it cannot reveal quantities that are not visible and is prone to rationalising small discrepancies. The comparison of a model's output with an analytic solution, the other classical technique, has so far not been applied to

generated simulators. Making it possible requires access to the simulator's internal state at arbitrary times, which motivates the minimal output contract of Section 3.

3 An executable oracle for generated mechanism simulators

3.1 Output contract

The generator receives a short prose description of the mechanism, for example "one leg of Theo Jansen's Strandbeest walking linkage, with the link lengths of Jansen's published holy numbers", together with a fixed output contract. It chooses all dimensions, placement, speed and direction itself. The contract asks for:

1. one self-contained HTML page that loads `three.js` from a fixed path and no other network resource;
2. one named `Object3D` per listed part (for example `crank`, `coupler`, `rocker`), holding all of that part's meshes, and a child marker named `<part>@<point>` at each listed joint of that part (`crank@A`, `coupler@A`, ...);
3. a panel element per listed quantity, marked with a `data-q` attribute (for example the crank angle and the foot position);
4. an object `window.SIM` exposing the scene, the period, the parameters the simulator actually uses (`SIM.params`: link lengths, pivot positions, angular speed, initial angle) and a function `render(t)` that poses all parts and the panel for time t. The function must be *pure*: no randomness, no clock, no state carried between calls.

The contract fixes *names*, not dimensions. It is a reduced version of the teardown skill's own contract [8], which already requires a pure `render(t)` so that videos can be captured frame by frame. In a first phase we also tried a detailed specification that fixed coordinate frames, assembly branches and formulas; it is discussed in Section 4.

3.2 Sampling

The oracle loads the page in headless Chrome through Playwright and waits for a readiness flag. It then calls `render(t)` at 120 instants per input period (two periods for the Geneva drive) and records the world position of every marker and the text of every panel element. Times are offset by a small irrational fraction of the period to avoid special angles. Tolerances scale with the machine: lengths are compared to within 1 mm per 100 mm of machine size (at least 1 mm), angles to within 1°, and relative quantities to within 1%.

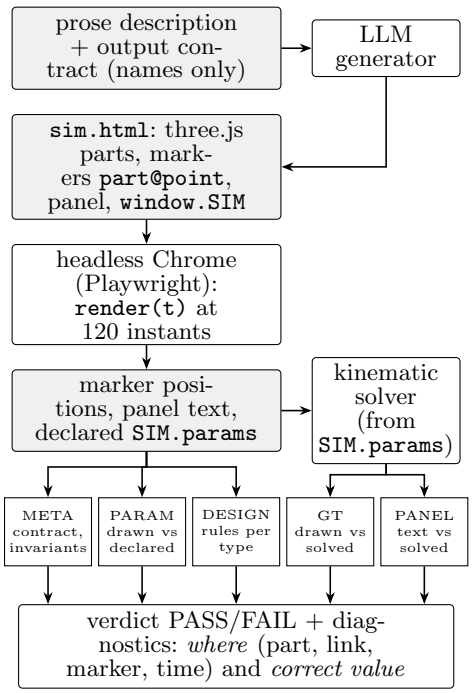


Fig. 1 The executable oracle. The generator chooses all dimensions; the oracle samples the page's pure `render(t)`, compares what is drawn with what the page declares, applies per-type design rules, and solves the kinematics from the declared parameters, so no per-task reference is written by hand.

3.3 Check families

Figure 1 gives an overview and Table 1 lists the checks. They fall into five families, ordered by how much ground truth they need.

Two design choices make the oracle reusable without a specification per task. First, GT and PANEL compare the drawing with the solution of the simulator's *own* declared parameters, not with a fixed reference design. Any dimensions are acceptable as long as the drawing is consistent with them. Second, engineering knowledge enters only through DESIGN rules, one small rule set per mechanism type. The rules state what any correct

Table 1 Check families of the oracle. Only DESIGN encodes mechanism-specific engineering knowledge; GT and PANEL use the kinematic solution computed from the simulator's own declared parameters.

Family	Checks	Ground truth needed
META	determinism under reversed call order; periodicity (t vs. $t+T$); rigidity of every part (marker distances constant); coincidence of every shared joint; fixed frame static; markers rigidly attached to their part's geometry (drift)	none (metamorphic and structural relations implied by the contract)
PARAM	drawn distances between markers equal the declared link lengths; fixed pivots at declared positions; input angle equals $\theta_0 + \omega t$; declared period consistent with ω	the simulator's own `SIM.params`
DESIGN	Grashof condition; Geneva pin enters slots tangentially; planetary $Z_r = Z_s + 2Z_p$ and $(Z_s + Z_r)$ divisible by the number of planets; cam law (cycloidal rise–dwell–return–dwell), pressure angle $\leq 30°$, no undercut; six-bar assembles and transmission angles stay in $[30°, 150°]$; Jansen's published link lengths; Chebyshev lambda proportions; Peaucellier–Lipkin conditions	engineering knowledge, one rule set per mechanism
GT	every marker vs. the position obtained by solving the loop-closure equations from `SIM.params` (assembly branch taken from the drawing at the first sample, or the canonical branch where one is defined)	kinematic solver per mechanism type
PANEL	every displayed number vs. the same solution	as GT

instance must satisfy, for example that a lambda linkage has Chebyshev's proportions up to a uniform scale, regardless of the scale and placement the generator chose. A proportion check therefore uses the best uniform scale. With ratios r_i between drawn and published values, the minimax relative error is $(r_{\max} - r_{\min})/(r_{\max} + r_{\min})$.

3.4 Guarding against marker gaming

Because GT checks markers, a simulator could pass by placing each marker at the solved position while drawing the parts elsewhere. We observed one repair attempt that said it did this (Section 5). To close this gap, the META family includes a drift check. For every marker it computes the marker's position in the local frame of each mesh of its part over 24 samples per period. It fails if, for every mesh, this position moves by more than 0.5 mm (scaled). A mutant that redraws a four-bar's coupler 0.3 rad off while resetting its marker passes all other checks but fails this one with a drift of 41 mm.

3.5 Diagnostics and feedback levels

For every failed check the oracle records a diagnostic that *localises* the problem and, where a ground truth exists, gives the *correct value*. Examples:

- *DESIGN, Jansen*: "Links that do not match at this scale: `len.XV` = 55.8 mm, expected

40.1 mm (looks swapped with `len.YV`); ...". The scale used for this diagnosis is the one at which most drawn lengths equal some published length, which is robust to permuted links.

- *DESIGN, Peaucellier*: "`len.OA` = 45 mm does not fit the inversor geometry — the crank must be exactly as long as the distance between the fixed pivots O and Q (expected 50 mm)".
- *GT*: "at $t = 2.016$ s (crank 181.4°) marker `rodUpper@Y` is drawn at $(-26.1, 38.8)$; solving the linkage from your SIM.params puts Y at $(-22, -39.2)$".
- *META*: "the markers `coupler2@D`, `rocker2@D` are 51.8 mm apart; parts sharing joint D must meet there at every instant".

From the same oracle result we derive three feedback levels by deterministic transformation, so that they differ only in information content:

*Location + value (*spec*)* the full diagnostics above.
*Location only (*locate*)* the same diagnostics with every correct value removed. Published numbers, "expected" values, solved positions and correct panel values are dropped, and for relation-type rules the relation is dropped too, so the message reduces to "`len.OA` does not fit the inversor geometry". What the simulator currently draws, the names of the wrong elements and "looks swapped with" hints are kept.
*Check + magnitude (*coarse*)* only the family, name, measured value and tolerance of each failed check, plus a one-line explanation of what proportion checks compare.

The oracle, including the eight mechanism modules, the specifications and the browser harness, is about 1,060 lines of JavaScript; it runs in 0.2 s per simulator (median) and uses no model calls.

4 Study design

4.1 Mechanisms

We use eight planar mechanisms chosen to span the error classes of Section 2. Four of them have textbook kinematics: a four-bar crank-rocker, an external Geneva drive with a free choice of 4 to 6 slots, a planetary gear set with a fixed ring and three planets, and a disk cam with an offset translating roller follower and a cycloidal rise–dwell–return–dwell law. The Watt II six-bar adds a second loop and transmission-angle limits. Three carry published design knowledge: Jansen's leg (published link lengths), Chebyshev's lambda linkage (published proportions) and the Peaucellier–Lipkin linkage (geometric relations). For each, a hand-written reference simulator passes the oracle; the largest residual is below 3×10^{-13} mm for the linkages and gears and 5×10^{-4} mm for the cam, whose roller–cam contact is measured against mesh vertices. For the lambda linkage we confirmed the published proportions numerically: the tracing point stays within 0.63% of the crank length from a straight chord for 200° of each revolution, consistent with the source [22]. For Peaucellier–Lipkin, the output point's coordinate across the line is constant to 10^{-13} mm over the whole swing.

4.2 Generators, reviewers and repairers

Except for phase 8, all generation, review and repair was done by two models of Anthropic's Claude family, a larger one (*Sonnet*, model identifier `claude-sonnet-5-5`) and a smaller one (*Haiku*, `claude-haiku-5-5`), with default sampling settings, between 8 and 13 November 2026. They were run as sub-agents of Claude Code, each with a fresh context and a one-line instruction to read a task file and follow it. The generation prompt is the mechanism description plus the output contract; it is published with the data. For a small generation-only check with another vendor (phase 8 below) we used two DeepSeek models, `deepseek-v4-pro` and `deepseek-flash`, through their chat API on 13 November 2026, with default sampling and the same prompt as a single user message. The API answer was written to the same file path that a sub-agent would have written.

Generation

Five samples per mechanism and model from the free-form ("spec-lite") description, 80 simulators in total. In an earlier phase, a detailed specification that fixed coordinates, branches and formulas for four textbook mechanisms (slider-crank, four-bar, two-stage gear train, Geneva) produced 8 simulators.

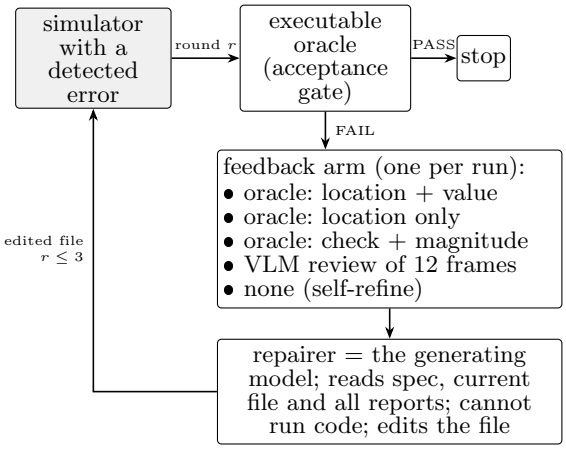


Fig. 2 Iterative repair. Oracle arms stop when the oracle passes; the VLM arm stops when the reviewer says "correct" and self-refinement when the repairer replies `NO_CHANGE`. The oracle grades every arm's final file, but only the oracle arms see its verdict.

Mutants

To measure detection independently of natural error rates, we seeded faults into reference simulators. There are 24 mutants for the detailed-specification mechanisms (six operators: reversed direction, link 10–15% too long, 15° phase offset, wrong assembly branch, wrong panel value, state carried between frames) and 15 for the spec-lite mechanisms (harmonic instead of cycloidal law, cam profile for the wrong rotation direction, base circle too small, lift offset, locked or branch-flipped six-bar, transmission angle out of range, swapped or wrong lambda proportions, Peaucellier conditions violated, drawn point off its marker). There is also one "gamed" mutant whose marker is reset to the solved position while its part is drawn elsewhere.

VLM reviewer

The VLM baseline is the larger model looking at a contact sheet of 12 rendered frames spread over one cycle. It is given the mechanism description and asked whether the simulator is kinematically correct and, if not, what is wrong. Inputs were anonymised (hashed identifiers, key kept separately) so that the reviewer could not infer the source. For the 24 detailed-specification mutants we ran the review twice, with and without the numeric panel visible.

Repair

Every simulator with an oracle-detectable natural error enters an iterative repair loop (Fig. 2). The repairer is the model that generated the file. In each round it reads the specification, the current file and all feedback reports so far, edits the file in place and replies `CHANGED` or `NO_CHANGE` with a one-line reason. It may not run code or read any other file. This isolates the information it receives through feedback, at the cost of disadvantaging self-refinement relative to an agent that can execute. Five feedback arms are compared:

- the three oracle levels of Section 3.5 (*spec*, *locate*, *coarse*), each stopping when the oracle passes;
- *VLM in the loop*: a fresh, blinded VLM review of the current version, stopping when the VLM says the simulator is correct;
- *self-refine*: a generic "a reviewer suspects errors, re-check against the specification" prompt, stopping when the repairer replies `NO_CHANGE`.

All arms have a budget of $K = 3$ rounds. Every version in every arm is graded by the oracle, whether or not the arm shows the grade to the repairer, so final correctness is measured the same way for all arms.

4.3 Phases and repetitions

The study evolved in phases, each motivated by the previous one. All data are reported.

1. Detailed specification, 4 mechanisms, 8 simulators and 24 mutants: no natural errors.
2. Spec-lite, 4 mechanisms (four-bar, Geneva, Jansen, planetary), 2 samples per cell.
3. One round of repair on the 7 faulty files with oracle, VLM or no feedback.
4. Iterative repair ($K = 3$) on the same files.
5. The full design: cam and Watt six-bar added, 5 samples per cell (60 simulators), and iterative repair repeated twice. Repetition 1 covers the 8 newly faulty files; repetition 2 covers those 8 plus the 7 files of phases 2–4.
6. The *locate* arm on exactly the file-runs of phase 5.
7. Chebyshev lambda and Peaucellier–Lipkin added (20 simulators) and iterative repair with the four non-VLM arms.

8. Generation only, other vendor: the two DeepSeek models on Jansen and lambda, 5 samples per cell (20 simulators). No repair was run.

Phases 3 and 4 shaped the design (they showed that one round is too little to separate the arms and that feedback must be localised), and we report them only briefly. The main repair analysis uses the 22 file-runs of phases 5–6 and the 6 of phase 7.

4.4 Measures and statistics

For RQ1 the unit is a generated simulator. Every oracle failure was inspected by hand to confirm that the error is real; Section 5 also reports one error that the oracle cannot see. For RQ2 the unit is a simulator or mutant, and we report true and false positives of the oracle and the VLM. For RQ3 and RQ4 the unit is a *file-run*, a faulty file in one repetition, and the outcome is whether its final version passes the oracle. All arms are applied to the same file-runs, so we compare them as paired binary outcomes: Cochran's Q across arms, exact McNemar tests per pair with Holm correction, Wilson 95% intervals for proportions, and Fisher's exact test between models. Repetition 2 contains the files of repetition 1, so file-runs are not fully independent. We therefore repeat the main tests with each source file counted once.

4.5 Use of large language models

Following the journal's policy, we state how LLMs were used. They are the *object* of study: they generated, reviewed and repaired the simulators as described above. Claude Code (with `claude-opus-5-5`) was also used as a programming assistant to implement the oracle, the reference simulators, the mutants and the analysis scripts, under the author's direction. Every oracle failure on generated code was checked by hand, the reference simulators were verified numerically, and the statistics were spot-checked by hand calculation. The same assistant helped draft and edit the manuscript text and searched the related literature; the author checked every number against the data and every reference against its source, and takes full responsibility for the content.

Table 2 Natural errors: simulators generated from the free-form specification that fail the oracle (5 samples per cell)

Mechanism	Design knowledge checked	Haiku	Sonnet
Four-bar crank-rocker	Grashof condition	0/5	1/5
Geneva drive	tangential pin entry	0/5	0/5
Planetary gear set	assembly condition	2/5	0/5
Disk cam, roller follower	motion law, pressure angle	0/5	0/5
Watt II six-bar	transmission angle	1/5	1/5
Jansen leg	published link lengths	5/5	4/5
Chebyshev lambda	published proportions	5/5	1/5
Peaucellier–Lipkin	geometric relations	0/5	0/5
Total		13/40	7/40

5 Results

5.1 RQ1: errors in generated simulators

Detailed versus free-form specifications

With the detailed specification, which fixed coordinates, branches and formulas, all 8 generated simulators passed the oracle (0/8, Wilson 95% CI 0–32%). With the free-form description of the same kind of mechanism, errors appeared at once: 7 of the first 16 simulators were wrong (44%, CI 23–67%; Fisher's exact test against the detailed specification $p = 0.054$). Once the model has to *choose* the dimensions and the assembly, errors move from writing kinematics code to recalling and applying design knowledge.

Error rates per mechanism

Table 2 shows the 80 free-form simulators. Twenty fail the oracle: 13 of 40 by the smaller and 7 of 40 by the larger model. On the Jansen leg, 9 of 10 simulators were wrong; on Chebyshev's lambda linkage, 6 of 10. Elsewhere, only 5 of 60 were wrong (8%).

All 20 failures were confirmed by hand. They fall into four classes:

- *Misremembered design knowledge* (15). In all nine faulty Jansen legs the generator recalled

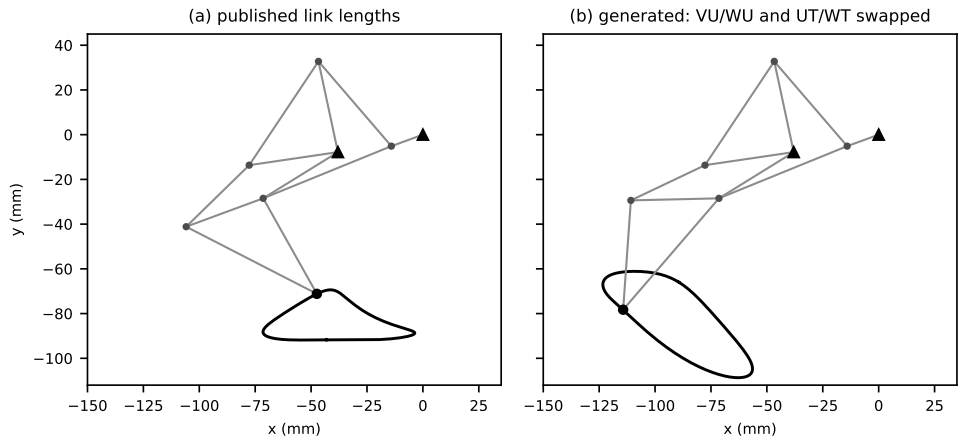


Fig. 3 Jansen leg solved from (a) the published link lengths and (b) the `SIM.params` of a simulator generated by the larger model, which swapped `VU/WU` and `UT/WT`. Grey: linkage at one crank angle; black curve: path of the foot T over one crank revolution; triangles: fixed pivots. Both pages run and animate without errors.

the published numbers but assigned some of them to the wrong links, or scrambled them, for example a crank of 38 instead of 15. Figure 3 shows a leg with two swapped pairs: the page runs, animates and looks like a Strandbeest, but the foot no longer has a flat ground-contact segment.With the lengths of the first two faulty legs, the flat part of the foot path shrank from 36% to 6% of the cycle and the stride from 49 to 11 mm. All five lambda linkages from the smaller model used ordinary four-bar proportions instead of Chebyshev's, and the larger model once placed the tracing point at the middle of the coupler instead of its extended end. A direct check of path straightness agrees with the proportion check on all ten lambda simulators: the four that pass trace a near-straight segment (1% of the crank length) over 200°, and the six that fail have no such segment. No correct alternative design was rejected.

- *Violated assembly conditions* (4). Two planetary sets had $Z_s + Z_r = 100$, not divisible by three, so three equally spaced planets cannot mesh with both sun and ring. Two Watt six-bars had a second loop that cannot close for part of the cycle.
- *Runtime failure* (1). A four-bar threw an exception on load because it assigned to a read-only property.
- *Geometry outside the contract* (1, not in the table). In one Geneva drive the cut-out of the locking disc faced away from the pin, so the wheel is not locked during dwell and the disc overlaps the wheel during indexing. The oracle cannot see this, because it checks markers, not the outline of the drawn parts; the VLM reviewer found it (Section 5.2).

Mechanisms whose design rule is a relation that the description implies produced no errors at all. This holds for the cam (0/10), whose motion law the description states, and for the Peaucellier–Lipkin linkage (0/10), whose conditions the models stated correctly in every case. The difference between models is clearest on the lambda linkage: 5/5 wrong for the smaller model versus 1/5 for the larger ($p = 0.048$).

Another vendor

The two DeepSeek models of phase 8 show the same pattern on the two design-knowledge mechanisms. One of the 20 replies was cut off at the output limit and contained no page, so 19 simulators ran. The larger model was wrong on 9 of 10 (Jansen 5/5, lambda 4/5) and the smaller on 5 of 9 (Jansen 4/4, lambda 1/5). Every failure was a DESIGN failure of the proportion check, and inspection confirmed each one. As with Claude, the Jansen legs used the published numbers but assigned some of them to the wrong links. None of the five faulty lambda linkages had Chebyshev's proportions. Here the relation between models is reversed: the larger DeepSeek model erred more often on lambda than the smaller. Error rates on recalled design knowledge are therefore high across vendors, but which model is worse is not stable.

5.2 RQ2: what the oracle and the VLM detect

Seeded mutants

The oracle detected all 24 detailed-specification mutants, all 15 spec-lite mutants, and, with the drift check, the gamed mutant. On the 24 detailed-specification mutants, checks that need no ground truth (META and structural checks) already caught 17: all state, length and phase faults, three of four branch faults and two of four direction faults. The remaining seven, a four-bar on the crossed branch, two reversed directions and four panel faults, need the kinematic solution. The VLM reviewer detected 20 of 24 mutants when the numeric panel was visible and 14 of 24 without it. Its detection of 10–15% length errors fell from 4/4 to 1/4. From its explanations, the VLM recovered lengths mainly by reading the panel and solving the kinematics itself, not by inspecting the drawing. Paired by mutant, the oracle's advantage is significant against the VLM without panel (10 versus 0 discordant mutants, exact McNemar $p = 0.002$) but not against the VLM with panel (4 versus 0, $p = 0.125$). Among the four mutants the VLM missed with the panel visible, it saw a 12–20° slot offset of the Geneva wheel but attributed it to perspective.

Natural errors

On the 15 renderable simulators of phase 2, the VLM found 5 of the 6 oracle-detected errors and reported 1 false positive on 9 correct simulators. It missed the planetary assembly violation, which is invisible in a rendering, but found the Geneva disc error that the oracle cannot see. On the 44 new simulators of phase 5 it had 7 true positives, 1 false negative (again a planetary tooth-count violation) and 4 false positives, all Geneva drives. The marker data show these wheels index by exactly 72° per driver turn; the reviewer misread intermittent motion as a fault. The VLM was also unstable: a fresh review of the same unchanged files at the start of repetition 2 called three faulty simulators correct, among them the Geneva drive it had flagged before.

The two oracles are complementary. The executable oracle catches errors of design knowledge and invisible errors (assembly conditions, state, panel values) deterministically, in about 0.2 s and with no model calls. The VLM catches shape errors outside the marker contract but misses invisible ones, produces false alarms on intermittent motion, and costs about 52,000 tokens per review.

Table 3 File-runs passing the oracle after each repair round, phases 5–6 pooled (22 file-runs; round 0 = generated file)

Arm	r0	r1	r2	r3
location + value	0	16	22	22
location only	0	12	14	14
check + magnitude	0	8	10	10
VLM review in the loop	0	5	5	5
Self-refine (no feedback)	0	6	6	6

5.3 RQ3: the oracle as an acceptance gate for repair

In two preliminary phases, one round of repair fixed 3 of 7 faulty files whether the feedback came from the oracle (as check names and magnitudes), the VLM or nobody. With three rounds, specific oracle feedback fixed 6 of 6 against 4/6 (coarse oracle), 2/6 (VLM) and 0/6 (self-refine). Tables 3 and 4 report the full design.

Over the 22 file-runs of phases 5–6, feedback with location and correct value fixed all 22, all within two rounds. It was followed by location only (14), check and magnitude (10), self-refinement (6) and VLM in the loop (5). The arms differ (Cochran's $Q = 44.5$, $df = 4$, $p = 5 \times 10^{-9}$). After Holm correction, full oracle feedback beats every other arm (location only $p = 0.047$, coarse $p = 0.004$, VLM $p = 1.5 \times 10^{-4}$, self-refine $p = 2.7 \times 10^{-4}$), and location-only beats VLM ($p = 0.027$) and self-refinement ($p = 0.047$). Location-only versus coarse (4 versus 0 discordant file-runs) and coarse versus VLM or self-refinement are not significant. With each source file counted once (20 file-runs including lambda, four arms) the conclusions do not change: full feedback 20/20 versus location only 10/20 (Holm $p = 0.008$), coarse 8/20 ($p = 0.002$) and self-refinement 4/20 ($p = 1.8 \times 10^{-4}$).

The difficulty splits by error class. Assembly and runtime errors were repaired with any oracle feedback (8/8 in all three oracle arms) and often even without (self-refinement 6/8, VLM 5/8); the arms do not differ significantly on this class (Q test

Table 4 Iterative repair (at most three rounds): file-runs whose simulator passes the oracle at the end, by feedback arm. Assembly/constraint errors: Watt six-bar, planetary, four-bar crash. Design knowledge: Jansen (phases 5–6) and Chebyshev lambda (phase 7). [a]The VLM arm was not run for the lambda file-runs, which are excluded from its counts.

		Design knowledge		
Feedback arm	Assembly / constraint	Haiku	Sonnet	All
Oracle, location + value	8/8	13/13	7/7	28/28
Oracle, location only	8/8	0/13	7/7	15/28
Oracle, check + magnitude	8/8	0/13	3/7	11/28
VLM review in the loop	5/8	0/8[a]	0/6[a]	5/22[a]
Self-refine (no feedback)	6/8	0/13	1/7	7/28

$p = 0.064$). Design-knowledge errors were repaired almost only with oracle feedback, as RQ4 details.

The two loops without an executable gate failed in characteristic ways. VLM-driven loops stopped 12 times on a "correct" verdict while the oracle still failed the file, and the VLM never recognised the planetary tooth-count error, so its arm did worse than self-refinement even on assembly errors. Self-refinement stopped when the repairer answered `NO_CHANGE`, and all 17 such answers on faulty Jansen legs declared the file correct; 10 of them stated explicitly that the link lengths "match Jansen's holy numbers". In one case the repairer stated that nothing was wrong with a four-bar that did not load.

5.4 RQ4: how specific must feedback be?

Pooling the design-knowledge file-runs (Jansen, phases 5–6, and lambda, phase 7), the result depends strongly on the repairing model (Table 4):

- The smaller model fixed 13 of 13 with location and value and 0 of 13 with location only, coarse feedback or no feedback (Cochran's $Q = 39$, $p = 1.7 \times 10^{-8}$; each pair with the full arm exact McNemar $p = 2.4 \times 10^{-4}$, Holm $p = 0.001$; counting each source file once, 10/10 versus 0/10, Holm $p = 0.012$).
- The larger model fixed 7 of 7 with location and value *and* 7 of 7 with location only, 3 of 7 with coarse feedback and 1 of 7 without feedback.
- With location only, the two models differ sharply (7/7 versus 0/13, Fisher $p = 1.3 \times 10^{-5}$); with location and value they do not (7/7 versus 13/13, $p = 1$).

The lambda linkage, added to test whether this pattern was specific to Jansen, reproduced it. The smaller model fixed 5/5 with values and 0/5 in each of the three other arms; the larger model's single faulty file was fixed in every arm, including self-refinement.

The repairers' replies show why location is not enough for the smaller model. The "looks swapped with" hints let it undo permutations. In two lambda files this left a single wrong link, the distance to the tracing point, which has to be recalled (five crank lengths). It never recovered that value in three rounds. Instead it substituted other proportions it believed to be classical (several different sets across files and rounds, none correct), back-computed a length from the reported error magnitude (four attempts, all wrong, since the minimax formula is not in the feedback), or answered `NO_CHANGE`. On Jansen, 9 of the 40 location-only attempts were `NO_CHANGE` replies claiming that the lengths matched the published numbers and that the checker's mapping must be wrong. The larger model, given only the names of the wrong links, supplied the published values from memory. With coarse feedback, it twice inferred the wrong Jansen link from the error magnitude itself, the only route by which coarse feedback fixed a Jansen leg.

5.5 Reliability of the oracle

The reference simulators pass, and every seeded mutant is detected. Every oracle failure on a generated simulator was confirmed by hand, so we observed no false positives. The single false negative is the Geneva disc of Section 5.1, which lies outside the contract. To exclude passes obtained by moving markers instead of parts, we audited

all passing generated simulators and all passing final repaired versions, 146 files, for marker drift: all had a drift of zero. One repairer explicitly tried this tactic, saying it had set markers "directly from the solved points", but its version still failed the design check. After we added the drift check and changed the Jansen scale estimate, we regraded all 661 earlier verdicts. 660 were unchanged, and the gamed mutant moved from pass to fail. Rule violations by agents were rare. One of 155 phase-5 repair attempts ran a Python command despite the instruction, and one phase-7 attempt stopped without writing a reply, asking instead to be told the classical proportions.

6 Discussion

An executable gate, not a replacement for review

The oracle and the VLM catch different errors (RQ2). The oracle is deterministic, explains its verdicts and costs nothing per run, but it only sees what the contract exposes. The VLM sees the drawn shapes but cannot see invisible conditions, misreads intermittent motion, and changes its verdict between identical runs. In a generation pipeline we would therefore use the oracle as the *acceptance gate* and repair driver, and keep a visual review as an additional, advisory check for shape errors outside the contract. In V&V terms, this puts analytic comparison first and animation second [9]. Our data show that a VLM alone as gate is not safe: 12 of its stop decisions accepted simulators that the oracle rejected.

Feedback should carry the reference value when one exists

Iterative repair with test feedback is well established [23–25], and the quality of feedback is known to limit self-repair [25]. Our ablation isolates one dimension of that quality, *information content*, at fixed source and wording, and shows that its effect depends on the model. Localisation alone sufficed for the larger model, which held the missing design knowledge, but did nothing for the smaller model, which did not. For a mechanism with published dimensions the oracle knows the correct value anyway, so withholding it has no benefit. More generally, an oracle that encodes reference knowledge should return that knowledge in its diagnostics, because the cheapest repairer may need it. This differs from feedback designs that report only failing tests or error magnitudes. It also refines earlier results: feedback lets weaker models match stronger ones [26], and its benefit depends on which model fills each role and on the task [27], but the information in the feedback was not varied at fixed source. The strongest feedback in ConvCodeWorld is written with access to the reference solution [26], which is consistent with our finding. In our setting it was the *kind* of information that mattered, not its amount.

Confident wrong knowledge

The most consistent failure we saw was not an inability to edit code. It was a repairer that defended a wrong design with a confident and specific claim ("the lengths match the published holy numbers"), sometimes blaming the checker. Self-correction without external feedback is known to be unreliable [28, 29], and LLMs hallucinate domain facts in code [30, 31]. Our data add that in an engineering domain such claims are cheap to falsify mechanically, but only if the acceptance step does not depend on the same model's judgement.

When is an oracle worth writing?

The oracle needed one module per mechanism type (27–90 lines of JavaScript each), written once and reused for every generated instance regardless of its dimensions. The checks that need no ground truth (META) already caught most seeded faults (17 of 24), and they transfer to any simulator that follows the contract. Errors were concentrated where design knowledge must be recalled rather than derived. Mechanisms specified by a relation or law that the description implies (cam, Peaucellier–Lipkin) produced no natural errors, whereas mechanisms that depend on remembered numbers (Jansen, lambda) produced most of them. A practical heuristic follows: invest in DESIGN rules for mechanisms whose function depends on published dimensions, and rely on the generic META, PARAM and GT checks elsewhere.

Leakage through magnitudes

Our coarse and location-only arms still reported the measured value of each failed check. The

smaller model repeatedly tried to invert this value into a corrected length, without success because the minimax formula was not shown. The larger model succeeded twice on Jansen. Error magnitudes are thus a side channel between feedback levels, and future ablations should control them explicitly.

7 Threats to validity

Construct validity

Correctness is defined by our oracle and by hand inspection by the author. The DESIGN rules encode engineering knowledge that we wrote. We verified the reference designs numerically (Jansen's foot path, the lambda straight segment, the Peaucellier line), but the rule set has not been reviewed by an independent mechanical engineer. Each rule is a textbook condition (Grashof, assembly of planetary sets, tangential Geneva entry, transmission and pressure angles) or a published dimension set with its source cited, and all rules are in the public repository for inspection. The marker contract is a precondition of the oracle and may change how models write simulators. Errors outside the contract, such as the Geneva disc, are invisible to it, so "passes the oracle" does not mean "correct" in every respect. The feedback levels are produced by transforming the oracle's text. Location-only and coarse feedback still contain the measured magnitude of each failed check, which a repairer can partly exploit (Section 6).

Internal validity

Repairers were not allowed to run code, so that all arms received information only through feedback. Self-refinement by an agent that can execute its own page might do better. Agents ran as subagents of Claude Code with that tool's system prompt, not as raw model calls. One of 155 phase-5 repair attempts ran a command despite the instruction; we kept it in the data. The design evolved over phases (Section 4), and the feedback levels were introduced after seeing phase-4 results. To limit the resulting risk of tailoring, phases 6 and 7 re-used the exact file-runs and stopping rules of phase 5 and added mechanisms chosen *before* their generations were seen.

External validity

The repair experiments used only two models of one vendor. A generation-only check with two DeepSeek models (phase 8) reproduced the RQ1 error pattern on Jansen and lambda, but it was small (19 runnable simulators) and called the models through their API rather than as Claude Code sub-agents. Repair with another vendor's model, and hence RQ3 and RQ4, is the most important missing replication. Because the oracle, prompts and feedback transformations are model-independent and published, it requires only new repair runs; the API runner used in phase 8 is included in the repository. The mechanisms are planar, single-input and textbook-sized. The significant RQ4 effect rests on two mechanisms with published dimensions (Jansen and lambda). The relation-type mechanism (Peaucellier–Lipkin) produced no natural errors, so it could not be tested on real faults. Simulators followed a reduced contract derived from one generation tool.

Conclusion validity

Samples are small (22 + 6 file-runs for repair). Repetition 2 contains the files of repetition 1, so file-runs are not independent. We report exact tests, Holm-corrected pairwise comparisons and an analysis that counts each source file once; the main effects (full oracle feedback versus all other arms; the model × feedback interaction) remain significant under it. Several pairwise differences, such as location-only versus coarse and oracle versus VLM with panel on mutants, are not significant, and we do not claim them.

8 Related work

LLM-generated 3D and physics code

Benchmarks for interactive scientific demonstrations [1], physics animations [2], physics-aware 3D scenes [32] and 3D worlds [7] show that generated programs usually run but often behave wrongly. WorldCoder-Bench also finds that screenshot-based VLM judging correlates poorly with state-level correctness. Generators of articulated objects and CAD models use a VLM critic in the loop [4–6], as does agentic generation of physics simulation code [3], which names the remaining "oracle gap". SGA verifies educational animations through their

scene graph [33]. Closest in domain are LLM-based linkage synthesis [34, 35], where a trusted simulator evaluates parameters the LLM proposes, and the generation of multibody models compared with expert ground truth [36]. In contrast, we test the simulator code itself, derive the ground truth from the simulator's declared parameters, and study the oracle as gate and feedback source.

Test oracles and metamorphic testing

Our oracle is a derived oracle in the sense of Barr et al. [12]. Its META family consists of metamorphic and structural relations [13, 14] implied by the output contract, analogous to metamorphic testing of graphics compilers [17], scientific software [15] and autonomous-driving systems [16]. Work on oracle automation with neural models [18–20] generates assertions. We instead encode reusable domain rules once per mechanism type and keep the oracle independent of the model under test, as recommended by Molina et al. [20]. Insufficient tests over-accept LLM code [21], and property-based tests have been proposed as a remedy [37]; our setting has no tests at all, so the oracle has to provide them.

Simulation V&V

Classical simulation V&V lists both animation and comparison with analytic results among validation techniques [9, 11, 38]. Our comparison of VLM review and executable oracle can be read as a comparison of these two techniques, automated, for generated simulators.

Repair with feedback

Self-Debug [23], Reflexion [39] and Self-Refine [40] use execution or self-generated feedback. Olausson et al. [25] show that feedback quality limits self-repair. Without an external signal, self-correction is unreliable [28, 29]. Among SE approaches, ChatRepair [24] repairs in conversation with test-failure feedback until tests pass, and Fan et al. [41] repair LLM-generated programs and find location information useful. ConvCodeWorld [26] varies the type and quality of feedback (compilation, execution with partial or full test coverage, novice or expert verbal feedback) in conversational code generation, and Pan et al. [27] vary model strength in the generator, feedback and refiner roles. To our knowledge, no prior study uses a single deterministic oracle to deliver feedback at controlled levels of information (location with value, location only, check with magnitude), compares them with VLM review and self-refinement under oracle-gated stopping, and reports an interaction between the needed information and the repairing model.

LLM and VLM judges

LLM judges are biased and unreliable for code correctness [42–44], and multimodal judges and VLMs struggle with simple geometry and visual glitches [45–47]. Our VLM results (unstable verdicts, false alarms on intermittent motion, invisible conditions missed) are consistent with this literature in a new domain.

In this journal

Related work in ISSE checks that a rendered block diagram matches its model [48], classifies AI for software engineering [49] and uses LLMs to interpret test logs of a digital twin [50]. We are not aware of an ISSE paper on oracles for LLM-generated simulation code.

9 Conclusion

Generated mechanism simulators run and look right much more often than they are right. Under free-form specifications, a quarter of the simulators in our study were wrong, mostly in design knowledge that the model had to recall. An executable oracle that derives its ground truth from the simulator's own declared parameters, plus a small set of design rules per mechanism type, detected these errors deterministically at negligible cost. Used as the acceptance gate of an iterative repair loop with diagnostics that name the faulty element and its correct value, it led to correct simulators in every faulty case within two rounds. VLM review and self-refinement stopped on wrong "correct" verdicts. How much the feedback must say depends on the repairing model: a larger model needed only the location of the error, a smaller one the correct value.

Future work includes replication with models from other vendors, mechanisms beyond planar linkages, contracts that expose part outlines so that shape errors become checkable, generating DESIGN rules from engineering references, and

controlling the magnitude side channel in feedback ablations.

Statements and Declarations

Funding. No funding was received for conducting this study.

Competing interests. The author has no relevant financial or non-financial interests to disclose.

Ethics approval. Not applicable; the study involves no human participants or personal data.

Data and code availability. The specifications, generated and repaired simulators, mutants, reference simulators, oracle, feedback texts, repair replies, raw oracle results and analysis scripts are available at https://github.com/haihpse150218/llm-mechsim-oracle (code under the MIT License, data and text under CC BY 4.0) and archived on Zenodo as version v1.1-submission, https://doi.org/10.5281/zenodo.24201916 (all versions: https://doi.org/10.5281/zenodo.24187716). The repository README explains how to regenerate every table and statistic in this paper (`pilot/stats/stats.mjs`, `paper/draft/make_tables.mjs`).

Author contributions. Phi Hai Hoang is the sole author and was responsible for conceptualisation, methodology, software, validation, formal analysis, investigation, data curation, visualisation, and writing (original draft, review and editing).

Use of generative AI. See Section 4, "Use of large language models".

References

[1] Chen, Q., Liu, Y., Li, L., Chen, K., Guo, Q., Cheng, G., Yuan, F.: InteractScience: Programmatic and visually-grounded evaluation of interactive scientific demonstration code generation. In: Proceedings of the 43rd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 306, pp. 15585–15613. PMLR, ??? (2026). https://proceedings.mlr.press/v306/chen26cf.html

[2] Wang, Y., Wang, R., Wang, Y., Liang, X., Koto, F., Baldwin, T., Liang, X., Li, H.: Training and Benchmarking Code Generation for Physics-Inspired Animations. arXiv:2602.10840; v1 titled "SimuScene: Training and Benchmarking Code Generation to Simulate Physical Scenarios" (2026). https://doi.org/10.48550/arXiv.2602.10840 . https://arxiv.org/abs/2602.10840

[3] Shende, P., Camburn, B.: Perceptual Self-Reflection in Agentic Physics Simulation Code Generation. arXiv:2602.12311 (2026). https://doi.org/10.48550/arXiv.2602.12311 . https://arxiv.org/abs/2602.12311

[4] Gao, Y., Ge, Y., Cai, P., Seita, D., Itti, L.: LAM: Language articulated object modelers. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16010–16020 (2026). https://openaccess.thecvf.com/content/CVPR2026/html/Gao_LAM_Language_Articulated_Object_Modelers_CVPR_2026_paper.html

[5] Le, L., Xie, J., Liang, W., Wang, H.-J., Yang, Y., Ma, Y.J., Vedder, K., Krishna, A., Jayaraman, D., Eaton, E.: Articulate-anything: Automatic modeling of articulated objects via a vision-language foundation model. In: The Thirteenth International Conference on Learning Representations (ICLR) (2025). https://arxiv.org/abs/2410.13882

[6] Alrashedy, K., Tambwekar, P., Zaidi, Z., Langwasser, M., Xu, W., Gombolay, M.: Generating CAD code with vision-language models for 3D designs. In: The Thirteenth International Conference on Learning Representations (ICLR) (2025). https://arxiv.org/abs/2410.05340

[7] Lu, S., Xu, Y., Yu, K., Jiang, S., Yu, Y., Wang, Y., Yang, H., Zhang, Y., Wang, B., He, R., Liang, J.: WorldCoder-Bench: Benchmarking physically grounded 3D world synthesis. In: Advances in Neural Information Processing Systems (NeurIPS) (2026). To appear; arXiv:2606.01869. https://arxiv.org/abs/2606.01869

[8] elvarofficial: teardown: interactive 3D machine simulators as single offline HTML pages (Claude Code skill). MIT licence

(2026). https://github.com/elvarofficial/teardown

[9] Sargent, R.G.: Verification and validation of simulation models. Journal of Simulation **7**(1), 12–24 (2013) https://doi.org/10.1057/jos.2012.20

[10] Jansen, T.: The Great Pretender. 010 Publishers, Rotterdam (2007). See also https://www.strandbeest.com

[11] Balci, O.: Verification, validation and accreditation of simulation models. In: Proceedings of the 29th Winter Simulation Conference (WSC), pp. 135–141 (1997). https://doi.org/10.1145/268437.268462

[12] Barr, E.T., Harman, M., McMinn, P., Shahbaz, M., Yoo, S.: The oracle problem in software testing: A survey. IEEE Transactions on Software Engineering **41**(5), 507–525 (2015) https://doi.org/10.1109/TSE.2014.2372785

[13] Chen, T.Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T.H., Zhou, Z.Q.: Metamorphic testing: A review of challenges and opportunities. ACM Computing Surveys **51**(1), 1–27 (2018) https://doi.org/10.1145/3143561

[14] Segura, S., Fraser, G., Sanchez, A.B., Ruiz-Cortés, A.: A survey on metamorphic testing. IEEE Transactions on Software Engineering **42**(9), 805–824 (2016) https://doi.org/10.1109/TSE.2016.2532875

[15] Kanewala, U., Bieman, J.M.: Testing scientific software: A systematic literature review. Information and Software Technology **56**(10), 1219–1232 (2014) https://doi.org/10.1016/j.infsof.2014.05.006

[16] Zhou, Z.Q., Sun, L.: Metamorphic testing of driverless cars. Communications of the ACM **62**(3), 61–67 (2019) https://doi.org/10.1145/3241979

[17] Donaldson, A.F., Evrard, H., Lascu, A., Thomson, P.: Automated testing of graphics shader compilers. Proceedings of the ACM on Programming Languages **1**(OOPSLA), 93–19329 (2017) https://doi.org/10.1145/3133917

[18] Dinella, E., Ryan, G., Mytkowicz, T., Lahiri, S.K.: TOGA: A neural method for test oracle generation. In: Proceedings of the 44th International Conference on Software Engineering (ICSE), pp. 2130–2141 (2022). https://doi.org/10.1145/3510003.3510141

[19] Hossain, S.B., Filieri, A., Dwyer, M.B., Elbaum, S., Visser, W.: Neural-based test oracle generation: A large-scale evaluation and lessons learned. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), pp. 120–132 (2023). https://doi.org/10.1145/3611643.3616265

[20] Molina, F., Gorla, A., d'Amorim, M.: Test oracle automation in the era of LLMs. ACM Transactions on Software Engineering and Methodology **34**(5), 1–24 (2025) https://doi.org/10.1145/3715107

[21] Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In: Advances in Neural Information Processing Systems 36 (NeurIPS), pp. 21558–21572 (2023). https://doi.org/10.52202/075280-0943

[22] Miler, D., Birt, D., Hoić, M.: Multi-objective optimization of the Chebyshev lambda mechanism. Strojniški vestnik – Journal of Mechanical Engineering **68**(12), 725–734 (2022) https://doi.org/10.5545/sv-jme.2022.349

[23] Chen, X., Lin, M., Schärli, N., Zhou, D.: Teaching large language models to self-debug. In: The Twelfth International Conference on Learning Representations (ICLR) (2024). https://openreview.net/forum?id=KuPixIqPiq

[24] Xia, C.S., Zhang, L.: Automated program repair via conversation: Fixing 162 out of 337

bugs for $0.42 each using ChatGPT. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 819–831 (2024). https://doi.org/10.1145/3650212.3680323

[25] Olausson, T.X., Inala, J.P., Wang, C., Gao, J., Solar-Lezama, A.: Is self-repair a silver bullet for code generation? In: International Conference on Learning Representations (ICLR) (2024). https://arxiv.org/abs/2306.09896

[26] Han, H., Hwang, S.-w., Samdani, R., He, Y.: ConvCodeWorld: Benchmarking conversational code generation in reproducible feedback environments. In: International Conference on Learning Representations (ICLR) (2025). https://arxiv.org/abs/2502.19852

[27] Pan, X., Phang, J., Davidson, G., Perez, E.: Generate-feedback-refine: How much does model quality in each role matter? In: 3rd Workshop on Deep Learning for Code (DL4C) at ICLR (2025). https://www.iclr.cc/virtual/2025/34833

[28] Huang, J., Chen, X., Mishra, S., Zheng, H.S., Yu, A.W., Song, X., Zhou, D.: Large language models cannot self-correct reasoning yet. In: International Conference on Learning Representations (ICLR) (2024). https://arxiv.org/abs/2310.01798

[29] Kamoi, R., Zhang, Y., Zhang, N., Han, J., Zhang, R.: When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs. Transactions of the Association for Computational Linguistics **12**, 1417–1440 (2024) https://doi.org/10.1162/tacl_a_00713

[30] Liu, F., Liu, Y., Shi, L., Yang, Z., Zhang, L., Lian, X., Li, Z., Ma, Y.: Beyond functional correctness: Exploring hallucinations in LLM-generated code. IEEE Transactions on Software Engineering **52**(3), 1037–1055 (2026) https://doi.org/10.1109/TSE.2026.3657432

[31] Tian, Y., Yan, W., Yang, Q., Zhao, X., Chen, Q., Wang, W., Luo, Z., Ma, L., Song, D.: CodeHalu: Investigating code hallucinations in LLMs via execution-based verification. Proceedings of the AAAI Conference on Artificial Intelligence **39**(24), 25300–25308 (2025) https://doi.org/10.1609/aaai.v39i24.34717

[32] Xie, T., Wang, P., Hu Jiaxin, Qian, Y., Wang, Y., Wang, S., Ma, R., Peng, Y., Wang, L., Tai, Y., Yang, J., Yi, Z.: PhysCodeBench: Benchmarking Physics-Aware Symbolic Simulation of 3D Scenes via Self-Corrective Multi-Agent Refinement. arXiv:2604.23580 (2026). https://doi.org/10.48550/arXiv.2604.23580 . https://arxiv.org/abs/2604.23580

[33] Lopez, J., Hinojosa, C., Ghanem, B.: SGA: Plug & play geometric verification for educational video synthesis. In: Findings of the Association for Computational Linguistics: EMNLP 2026, pp. 9533–9549. Association for Computational Linguistics, ??? (2026). https://doi.org/10.18653/2026.findings-emnlp.454

[34] Gandarela, J.P., Rios, T., Menzel, S., Freitas, A.: Symbolic Intermediaries as a Linguistic-Numerical Interface for LLM-Driven Geometric Reasoning. arXiv:2505.17607v3 (2026); v1 titled "Controlled Agentic Planning & Reasoning for Mechanism Synthesis" (2025). https://doi.org/10.48550/arXiv.2505.17607 . https://arxiv.org/abs/2505.17607

[35] Gandarela, J.P., Rios, T., Menzel, S., Freitas, A.: Language Models Refine Mechanical Linkage Designs Through Symbolic Reflection and Modular Optimisation. arXiv:2604.27962 (2026). https://doi.org/10.48550/arXiv.2604.27962 . https://arxiv.org/abs/2604.27962

[36] Gerstmayr, J., Weyrer, S., Möltner, T., Manzl, P., Pieber, M.: Large Language Models and their Awareness of Mechanics and Spatial Geometry. arXiv:2608.14615 (2026). https://doi.org/10.48550/arXiv.2608.14615 . https://arxiv.org/abs/2608.14615

[37] Vikram, V., Lemieux, C., Sunshine, J., Padhye, R.: Can Large Language Models Write Good Property-Based Tests? (2023). https://doi.org/10.48550/arXiv.2307.04346

[38] Kleijnen, J.P.C.: Verification and validation of simulation models. European Journal of Operational Research **82**(1), 145–162 (1995) https://doi.org/10.1016/0377-2217(94)00016-6

[39] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems 36 (NeurIPS), pp. 8634–8652 (2023). https://doi.org/10.52202/075280-0377

[40] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P.: Self-refine: Iterative refinement with self-feedback. In: Advances in Neural Information Processing Systems 36 (NeurIPS), pp. 46534–46594 (2023). https://doi.org/10.52202/075280-2019

[41] Fan, Z., Gao, X., Mirchev, M., Roychoudhury, A., Tan, S.H.: Automated repair of programs from large language models. In: Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1469–1481 (2023). https://doi.org/10.1109/ICSE48619.2023.00128

[42] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In: Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track, pp. 46595–46623 (2023). https://doi.org/10.52202/075280-2020

[43] Wang, P., Li, L., Chen, L., Cai, Z., Zhu, D., Lin, B., Cao, Y., Kong, L., Liu, Q., Liu, T., Sui, Z.: Large language models are not fair evaluators. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 9440–9450 (2024). https://doi.org/10.18653/v1/2024.acl-long.511

[44] Crupi, G., Tufano, R., Velasco, A., Mastropaolo, A., Poshyvanyk, D., Bavota, G.: On the effectiveness of LLM-as-a-judge for code generation and summarization. IEEE Transactions on Software Engineering **51**(8), 2329–2345 (2025) https://doi.org/10.1109/TSE.2025.3586082

[45] Chen, D., Chen, R., Zhang, S., Wang, Y., Liu, Y., Zhou, H., Zhang, Q., Wan, Y., Zhou, P., Sun, L.: MLLM-as-a-judge: Assessing multimodal LLM-as-a-judge with vision-language benchmark. In: Proceedings of the 41st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 235, pp. 6562–6595. PMLR, ??? (2024). https://proceedings.mlr.press/v235/

[46] Rahmanzadehgervi, P., Bolton, L., Taesiri, M.R., Nguyen, A.T.: Vision language models are blind. In: Cho, M., Laptev, I., Tran, D., Yao, A., Zha, H. (eds.) Computer Vision – ACCV 2024, Part V. Lecture Notes in Computer Science, vol. 15476, pp. 293–309. Springer, Singapore (2024). https://doi.org/10.1007/978-981-96-0917-8_17

[47] Taesiri, M.R., Feng, T., Bezemer, C.-P., Nguyen, A.: GlitchBench: Can large multimodal models detect video game glitches? In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 22444–22455 (2024). https://doi.org/10.1109/CVPR52733.2024.02118

[48] Waldvogel, A., Koehler, F., Annighoefer, B.: Model-based block diagram recognition for model visualization verification – extended version. Innovations in Systems and Software Engineering **22**(2), 19 (2026) https://doi.org/10.1007/s11334-026-00633-9

[49] Schieferdecker, I.K.: Augmenting software engineering with AI. the ai4se taxonomy and its use. Innovations in Systems and Software Engineering **22**(2), 29 (2026) https://doi.org/10.1007/s11334-026-00652-6

[50] Hermawan, R., Bucaioni, A., Enoiu, E., Afzal, W., Saadatmand, M., Zaimovic, N., Ibtasham, M.S.: Let's read the log: Root cause analysis of railway test execution logs

with large language models. Innovations in Systems and Software Engineering **22**(2), 33 (2026) https://doi.org/10.1007/s11334-026-00657-1